LangChain v1.2.x: Tools & Tool Calling

Making LLMs interact with the real world

LangChain Framework

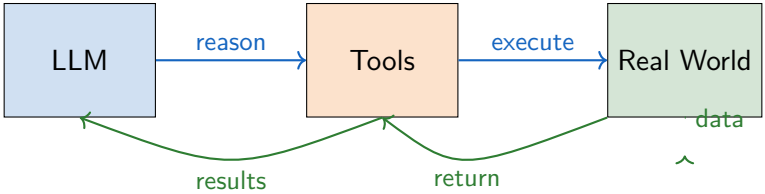
March 2026

Agenda

- Mental model: Tools as bridges to the real world
- Tool definitions: what is a tool?
- Creating tools: decorators and explicit schemas
- Tool schemas and model binding
- Tool calling flow and execution patterns
- Advanced: async, validation, MCP integration
- Common pitfalls and best practices
- Self-check quiz

By the end: you'll be able to create, bind, and invoke tools with LLMs

Mental Model: Tools as Bridges



Tools are the mechanism for LLMs to **perceive and act** on the world beyond text completion.

Key Insight

Without tools, LLMs can only talk about things. With tools, LLMs can *do* things.

What is a Tool? Core Definition

A Tool is...

A callable interface between an LLM and an external system, defined by:

- **Name**: unique identifier (e.g., `search_documents`)
- **Description**: human-readable explanation for the LLM
- **Schema**: input type, argument names, types, descriptions
- **Callable**: Python function that executes the tool
- **Return type**: what the tool outputs (typically str or structured)

Example Tool (conceptual)

```
Tool(name="get_weather", description="...", schema={...,
```
func=$\text{get}_{w}eather_{i}mpl$)

[Docs: langchain.tools.tool]

Tool Invocation Flow

1. LLM receives user query + bound tools
2. LLM **decides** which tool(s) to call (if any)
3. LLM returns `tool_calls`: list of {name, args}
4. Application **executes** the tool with args
5. Application returns `ToolMessage` with results
6. Optionally, LLM processes results and calls more tools

Critical Point

The LLM **never directly executes** the tool. It decides to call it; you execute it.

The @tool Decorator: Quick Start

Simplest Way to Create a Tool

```python
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city.

    Args:
        city: City name (e.g., 'New York')

    Returns:
        Weather description
    """
    # Simulated weather lookup
    return f"Weather in {city}: Sunny, 72 F"

# Use it
tool_obj = get_weather
print(tool_obj.name)  # "get_weather"
print(tool_obj.description)  # docstring used as description
```

[Docs: langchain.tools.tool]

Tool Arguments: Type Annotations Matter

Type Annotations Define the Schema

```python
from langchain.tools import tool

@tool
def search(
    query: str,
    limit: int = 10,
    include_metadata: bool = False
) -> str:
    """Search documents.

    Args:
        query: Search string
        limit: Max results (default 10)
        include_metadata: Include metadata in results
    """
    return f"Results for '{query}': 5 docs found"

# The LLM sees the schema and types
print(search.args_schema)  # JSON schema of args
```

Important

Type annotations are required! The LLM needs them to know how to call your tool.

Tool Descriptions: Docstrings

Good Docstring Practice

```python
@tool
def fetch_user_profile(user_id: str) -> str:
    """Retrieve user profile by ID.

    Use this when you need user contact info, preferences,
    or account status. Only call once per user ID.

    Args:
        user_id: The unique user identifier (UUID format)

    Returns:
        JSON string with name, email, status fields
    """
    # Implementation
    return '{"name":"Alice","status":"active"}'
```

Best Practice

Docstrings should explain **when** to use the tool, not just what it does. The LLM uses this to decide.

StructuredTool: Explicit Schema Definition

When `@tool` isn't explicit enough, use `StructuredTool`:

StructuredTool for Complex Schemas

```python
from langchain.tools import StructuredTool
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="Search")
    filters: dict = Field(default={})

def search_impl(query: str,
                filters: dict = None) -> str:
    return "Results..."

tool = StructuredTool.from_function(
    func=search_impl,
    name="semantic_search",
    description="Search documents",
    args_schema=SearchInput
)
```

[Docs: langchain.tools.StructuredTool]

Tool Schema: How LLMs See Your Tool

Schema Generated from Tool

```
# When you create a tool, it generates JSON schema
{
  "type": "object",
  "properties": {
    "query": {
      "type": "string",
      "description": "Search string"
    },
    "limit": {
      "type": "integer",
      "description": "Max results",
      "default": 10
    }
  },
  "required": ["query"],
  "description": "Search documents..."
}
```

This schema is sent to the LLM in the model call. The LLM uses it to decide: - What arguments to provide - What types they should be - Whether arguments are optional

[Docs: langchain.tools]

bind_tools(): Attaching Tools to a Model

Bind Tools to a Chat Model

```python
from langchain_anthropic import ChatAnthropic
from langchain.tools import tool

@tool
def get_time() -> str:
    """Get current time."""
    from datetime import datetime
    return datetime.now().isoformat()

model = ChatAnthropic(model="claude-3-5-sonnet-20241022")
model_with_tools = model.bind_tools([get_time])

# Now the model knows about get_time
response = model_with_tools.invoke(
    "What time is it?"
)
print(response.tool_calls)  # List of tool calls
```

[Docs: langchain.chat_models.BaseChatModel.bind_tools]

Tool Calling Flow: Complete Example

From Query to Tool Execution

```python
from langchain.tools import tool
from langchain_anthropic import ChatAnthropic

@tool
def weather(city: str) -> str:
    """Get weather."""
    return f"Sunny in {city}"

model = ChatAnthropic().bind_tools([weather])

# 1. User asks
response = model.invoke("What's weather in NYC?")

# 2. LLM returns tool_calls
print(response.tool_calls)
# Output: [{'name': 'weather', 'args': {'city': 'NYC'}, ...}]

# 3. Execute the tool
tool_result = weather.invoke({"city": "NYC"})

# 4. Feed back to model (optional chaining)
# Continue conversation with the result
```

ToolMessage: Returning Tool Results to LLM

Use ToolMessage for Proper Continuations

```python
from langchain.schema import ToolMessage, HumanMessage
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic().bind_tools([weather_tool])

# 1. Initial message
messages = [HumanMessage("Weather in NYC?")]

# 2. Get tool call
response = model.invoke(messages)
messages.append(response)

# 3. Execute and wrap in ToolMessage
tool_result = "Sunny"
messages.append(
    ToolMessage(
        content=tool_result,
        tool_call_id=response.tool_calls[0]["id"]
    )
)

# 4. Continue conversation
final = model.invoke(messages)
print(final.content)
```

[Docs: langchain.schema.ToolMessage]

Multiple Tools in One Call

LLM Can Call Multiple Tools at Once

```python
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get weather."""
    return "Sunny"

@tool
def get_time(timezone: str) -> str:
    """Get time."""
    return "15:30"

model = ChatAnthropic().bind_tools(
    [get_weather, get_time]
)

response = model.invoke(
    "What's weather in NYC and time in EST?"
)

# response.tool_calls can have multiple items
for call in response.tool_calls:
    print(f"Tool: {call['name']}, Args: {call['args']}")
```

Efficiency

Parallel tool calls reduce latency when multiple tools are needed.

Tool Choice: Controlling When Tools Are Called

Tool Choice Parameter

```python
model = ChatAnthropic()

# Auto (default): LLM decides if tool is needed
model_auto = model.bind_tools(
    [search_tool],
    tool_choice="auto"
)

# Required: LLM must call a tool
model_required = model.bind_tools(
    [search_tool],
    tool_choice="required"
)

# Forced: LLM must call specific tool
model_forced = model.bind_tools(
    [search_tool],
    tool_choice="search_tool"
)

response = model_forced.invoke("Hello")
# Will always return tool_calls for search_tool
```

[Docs: langchain.chat_models.BaseChatModel.bind_tools]

Built-in Tools: LangChain Provides Some

Using LangChain Built-in Tools

```python
from langchain.tools import (
    DuckDuckGoSearchRun,
    WikipediaQueryRun,
    ArxivQueryRun
)
from langchain_community.tools import (
    PythonREPLTool,
    ShellTool
)

# Ready-made tools
search = DuckDuckGoSearchRun()
wiki = WikipediaQueryRun()
python_repl = PythonREPLTool()

# Bind to model
model_with_tools = model.bind_tools(
    [search, wiki, python_repl]
)
```

Discovery

Check `langchain.tools` and `langchain_community.tools` for ready-made integrations.

Async Tools: Non-blocking Execution

Async Tool Implementation

```python
from langchain.tools import tool
import asyncio

@tool
async def fetch_from_api(url: str) -> str:
    """Fetch data from URL asynchronously."""
    # Simulated async I/O
    await asyncio.sleep(0.5)
    return f"Data from {url}"

# Invoke asynchronously
async def main():
    result = await fetch_from_api.ainvoke(
        {"url": "https://api.example.com"}
    )
    print(result)

asyncio.run(main())
```

Use Async for I/O-bound Tools

Network calls, database queries, file operations benefit from async.

Error Handling and Validation

Defensive Tool Implementation

```python
from langchain.tools import tool
from typing import Optional

@tool
def divide(a: float, b: float) -> str:
    """Divide two numbers safely."""
    if b == 0:
        return "Error: Division by zero"
    try:
        result = a / b
        return str(result)
    except Exception as e:
        return f"Computation error: {str(e)}"

# Return errors as strings, not exceptions
# The LLM sees the error and can retry or adjust
```

Best Practice

Always return error messages as tool output. Exceptions break the tool calling flow.

Input Validation and Sanitization

Validate Inputs Before Use

```python
from langchain.tools import StructuredTool
from pydantic import BaseModel, Field, validator

class QueryInput(BaseModel):
    query: str = Field(
        description="Search query",
        min_length=1,
        max_length=500
    )

    @validator('query')
    def query_not_harmful(cls, v):
        # Sanitize input
        if "DROP TABLE" in v.upper():
            raise ValueError("SQL-like queries not allowed")
        return v.strip()

def search_impl(query: str) -> str:
    # query is already validated
    return f"Results for {query}"

tool = StructuredTool.from_function(
    func=search_impl,
    args_schema=QueryInput
)
```

Pydantic validators execute before the function runs.

MCP Integration: Using MCP Tools with LangChain

What is MCP?

Model Context Protocol (MCP) is a protocol for exposing tools and resources. LangChain can adapt MCP servers to work as tools.

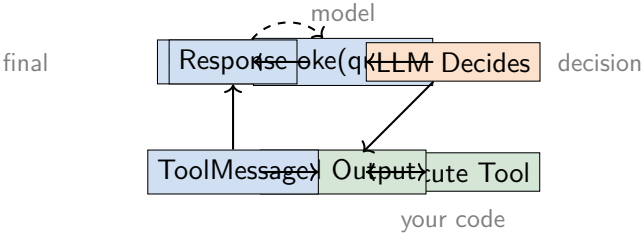
Using MCP Tools

```python
from langchain_core.tools import (
    convert_to_openai_tool
)
# Hypothetical MCP tool adapter
# (MCP ecosystem still evolving in March 2026)

# You can wrap MCP-exposed tools
mcp_tool = get_mcp_tool("filesystem")
openai_tool = convert_to_openai_tool(mcp_tool)
model_with_tool = model.bind_tools([openai_tool])
```

[Docs: MCP specification and LangChain integration]

Tool Calling Lifecycle (TikZ Diagram)



Loop back for follow-up tool calls (agentic behavior).

Real-World Example: Simple Agent Loop

Tool-Based Agent Loop

```python
from langchain.schema import HumanMessage, ToolMessage
from langchain_anthropic import ChatAnthropic
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """Search documents."""
    return "Found information about " + query

model = ChatAnthropic().bind_tools([search])
messages = [HumanMessage("What is LangChain?")]

while True:
    response = model.invoke(messages)
    if not response.tool_calls:
        print("Final:", response.content)
        break

    # Execute each tool call
    for call in response.tool_calls:
        tool_result = search.invoke(
            call['args']
        )
        messages.append(
            ToolMessage(content=tool_result,
                        tool_call_id=call['id'])
        )
    messages.append(response)
```

Pitfall #1: Missing or Vague Descriptions

The Problem

LLM doesn't know when to use your tool if description is missing or vague.

Bad: No Description

```python
@tool
def foo(x: str) -> str:
    # No docstring!
    return "result"

# LLM: "I don't know what foo does"
```

Good: Clear Description

```python
@tool
def retrieve_document(doc_id: str) -> str:
    """Retrieve full document by unique ID.

    Use when user asks for specific documents
    by ID or when you need the full content.
    """
    return "Document content..."
```

Pitfall #2: Missing or Wrong Type Annotations

The Problem

Without type hints, schema generation fails or becomes ambiguous.

Bad: No Type Hints

```
@tool
def process(data):
    # LLM: "What type is data? Can't call it"
    return data

# The function works but tool is unusable
```

Good: Full Type Hints

```
from typing import List

@tool
def process(data: List[str]) -> str:
    """Process a list of strings."""
    return ", ".join(data)

# Now LLM knows: pass list of strings, get string
```

Pitfall #3: Forgetting ToolMessage in Chains

The Problem

If you don't wrap tool results in ToolMessage, the LLM can't properly track which result belongs to which tool.

Bad: No ToolMessage

```
response = model.invoke(user_query)
tool_result = my_tool.invoke(response.tool_calls[0]['args'])

# Directly appending raw result breaks context
messages.append(tool_result)  # WRONG!
```

Good: Use ToolMessage

```
from langchain.schema import ToolMessage

tool_result = my_tool.invoke(call['args'])
messages.append(
    ToolMessage(
        content=tool_result,
        tool_call_id=call['id']
    )
)
```

Pitfall #4: Raising Exceptions Instead of Returning Errors

The Problem

Exceptions break the tool calling loop. Return errors as strings instead.

Bad: Raising Exceptions

```python
@tool
def risky_operation(x: int) -> int:
    if x < 0:
        raise ValueError("x must be positive")
    return x * 2

# Tool call fails, loop breaks
```

Good: Return Error Strings

```python
@tool
def risky_operation(x: int) -> str:
    if x < 0:
        return "Error: x must be positive"
    return str(x * 2)

# LLM sees error and can retry/adjust
```

Pitfall #5: Tools with Unintended Side Effects

The Problem

Avoid tools that modify state unexpectedly. LLM might call them multiple times.

Bad: Side Effects

```python
charge_count = 0

@tool
def charge_user(amount: float) -> str:
    global charge_count
    charge_count += 1
    # Actually charge the user
    # LLM calls twice = double charge!
    process_payment(amount)
    return "Charged"
```

Good: Idempotent

```python
@tool
def get_user_balance(user_id: str) -> str:
    """Read-only: Get balance."""
    return str(db.get_balance(user_id))

# Safe to call multiple times
```

Tool Composition with LCEL

Compose Tools in Chains

```python
from langchain_core.runnables import RunnablePassthrough
from langchain.tools import tool

@tool
def summarize(text: str) -> str:
    """Summarize text."""
    return text[:50] + "..."

@tool
def translate(text: str) -> str:
    """Translate to Spanish."""
    return "Translated: " + text

# Chain tools: pass output of one to next
chain = (
    RunnablePassthrough()
    | {"result": summarize}
    | lambda x: translate.invoke(x["result"])
)

output = chain.invoke("Long text here...")
```

Tools integrate naturally with LCEL chains.

Advanced: Retry Logic for Tools

Implement Tool Retries

```python
from tenacity import retry, stop_after_attempt

@tool
@retry(stop=stop_after_attempt(3))
def flaky_api_call(query: str) -> str:
    """Call an API that might fail."""
    import random
    if random.random() < 0.5:
        raise Exception("API timeout")
    return f"Result for {query}"

# Decorator handles retries transparently
result = flaky_api_call.invoke({"query": "test"})
```

Combine LangChain tools with libraries like Tenacity for resilience.

Monitoring and Logging Tool Calls

Log Tool Invocations

```python
import logging
from langchain.tools import tool

logger = logging.getLogger(__name__)

@tool
def logged_search(query: str) -> str:
    """Search with logging."""
    logger.info(f"Tool called: search, query={query}")
    result = "Results..."
    logger.info(f"Tool result: {result}")
    return result

# Or use LangChain callbacks (Section 10)
from langchain.callbacks import StdOutCallbackHandler
chain = model.bind_tools([logged_search])
chain.invoke(query, callbacks=[StdOutCallbackHandler()])
```

Track tool usage for debugging and analytics.

Return Types: What Tools Should Return

Return Types

```python
from typing import Dict, List
import json

@tool
def get_structured_data(id: str) -> str:
    """Return structured data as JSON string."""
    data = {"id": id, "values": [1, 2, 3]}
    return json.dumps(data)

@tool
def get_simple_text(query: str) -> str:
    """Return plain text."""
    return f"Answer: {query}"

# Always return str for LLM consumption
# Use JSON strings for structured data
```

Best Practice

Tools should return strings. Convert objects to JSON if needed.

Testing Tools Independently

Unit Test Your Tools

```python
from langchain.tools import tool
import pytest

@tool
def calculate_discount(price: float,
                       percent: int) -> str:
    """Apply discount to price."""
    if percent < 0 or percent > 100:
        return "Error: percent must be 0-100"
    result = price * (1 - percent / 100)
    return str(result)

def test_calculate_discount():
    result = calculate_discount.invoke({
        "price": 100,
        "percent": 10
    })
    assert result == "90.0"

    result = calculate_discount.invoke({
        "price": 100,
        "percent": 150
    })
    assert "Error" in result
```

Test tools like normal Python functions.

Tool Design Best Practices

1. **Clear descriptions**: Explain when to use the tool
2. **Type annotations**: All parameters must have types
3. **Good docstrings**: Include use cases in the docstring
4. **Error handling**: Return errors as strings, not exceptions
5. **Validation**: Use Pydantic for complex inputs
6. **Idempotency**: Safe to call multiple times
7. **Atomic**: One tool = one action
8. **Testability**: Tools should be testable in isolation
9. **Return strings**: LLMs consume text, use JSON for structured data
10. **Performance**: Keep tools fast; use async for I/O

Quick Reference: Tools Cheat Sheet

The @tool Decorator Way (Quickest)

```python
from langchain.tools import tool

@tool
def my_tool(arg1: str, arg2: int) -> str:
    """Description of what my_tool does."""
    return "result"

model = ChatAnthropic().bind_tools([my_tool])
response = model.invoke("Use my_tool")
```

The StructuredTool Way (Complex)

```python
from langchain.tools import StructuredTool
from pydantic import BaseModel

class Input(BaseModel):
    x: str

tool = StructuredTool.from_function(
    func=my_func, args_schema=Input
)
```

Self-Check: Do You Understand?

1. What's the difference between `@tool` and `StructuredTool`?
   - `@tool`: Decorator for simple tools from functions (infers schema from type hints)
   - `StructuredTool`: Explicit for complex tools with custom Pydantic schemas
2. What must you return after tool execution?
   - `ToolMessage(content=result, tool_call_id=...)`
3. Why do tools need descriptions?
   - LLM uses descriptions to decide when to call the tool
4. What happens if you raise an exception in a tool?
   - The tool calling loop breaks; return errors as strings instead

Self-Check: More Questions

5. How do you attach tools to a model?
   - Use `model.bind_tools([tool1, tool2, ...])`, returns a new model with tools bound
6. What's the purpose of `tool_choice="required"`?
   - Forces the LLM to always call a tool (useful for agentic loops)
7. Can tools be called asynchronously?
   - Yes, define them as `async def` and use `.ainvoke()`
8. What's an idempotent tool?
   - One that returns the same result when called multiple times; safe for LLM retries

Key Takeaways

Tools Enable Agency

Tools are how LLMs move from passive text generation to active interaction with the world.

Three Patterns

- **@tool**: Quick decorator for simple functions
- **StructuredTool**: Explicit schemas for complex tools
- **bind_tools()**: Attach tools to models for calling

Remember

LLM **decides** to call a tool, but **you** execute it and return ToolMessage with results.

What's Next?

- **Section 05: Agents**
  - Agentic loops that autonomously use tools to solve problems
  - Agent frameworks: ReAct, MRKL, conversational agents
- **Section 11: LangGraph**
  - Structured control flow for tool-using applications
  - State machines and multi-step reasoning
- **Section 10: Callbacks & Monitoring**
  - Track tool invocations, trace execution

Tools are the foundation for everything that follows!

References & Further Reading

- **Official LangChain Docs**
  - Tools: `https://python.langchain.com/docs/modules/tools/`
  - Tool usage: `https://python.langchain.com/docs/modules/tools/tool_use/`
- **LangChain API Reference**
  - `langchain.tools`: Decorators and classes
  - `langchain.schema`: Message types (ToolMessage)
- **Agent Patterns**
  - ReAct paper: Yao et al., 2022
  - Tool use with Claude: `https://docs.anthropic.com/`

[Docs: langchain.tools, langchain.schema.ToolMessage]

Conclusion

You now understand:

- What tools are and why they matter
- How to create tools with decorators and schemas
- How to bind tools to models
- The tool calling flow and how to execute tools
- Common pitfalls and how to avoid them
- Advanced patterns: async, validation, monitoring

You're ready to build agentic applications!